Resultados Preliminares e Análise de Desempenho

Lucas Fraga Balbinot, Matheus Augusto Tregnago, Rafael Silva de Souza

UFRGS — CMP223 — Análise de Desempenho

Ambiente de Teste

Execução realizada no **PCAD (UFRGS)** com diferentes configurações:

- **1 GPU (baseline)** → execução sem comunicação
- **2 GPUs (TP e PP)**
- **4 GPUs (TP e PP)**

Nós utilizados:

- Tupi (principal)
- Poti (distribuído)
- Experimento adicional em máquina única

Ajuste de Modelo

- Modelo original (**Llama**) não coube em algumas GPUs
- Substituído por:

➡️
Qwen2.5-7B-Instruct

Motivo:

- Menor footprint de memória
- Permitiu execução em mais configurações
- Manteve representatividade do problema

Organização dos Experimentos

Ordem adotada para análise:

1. **Single GPU (baseline)**
2. **Tensor Parallelism (TP)**
3. **Pipeline Parallelism (PP)**

Objetivo:

- Comparar impacto incremental da comunicação
- Isolar overhead introduzido por paralelismo

Resultados — Visão Geral

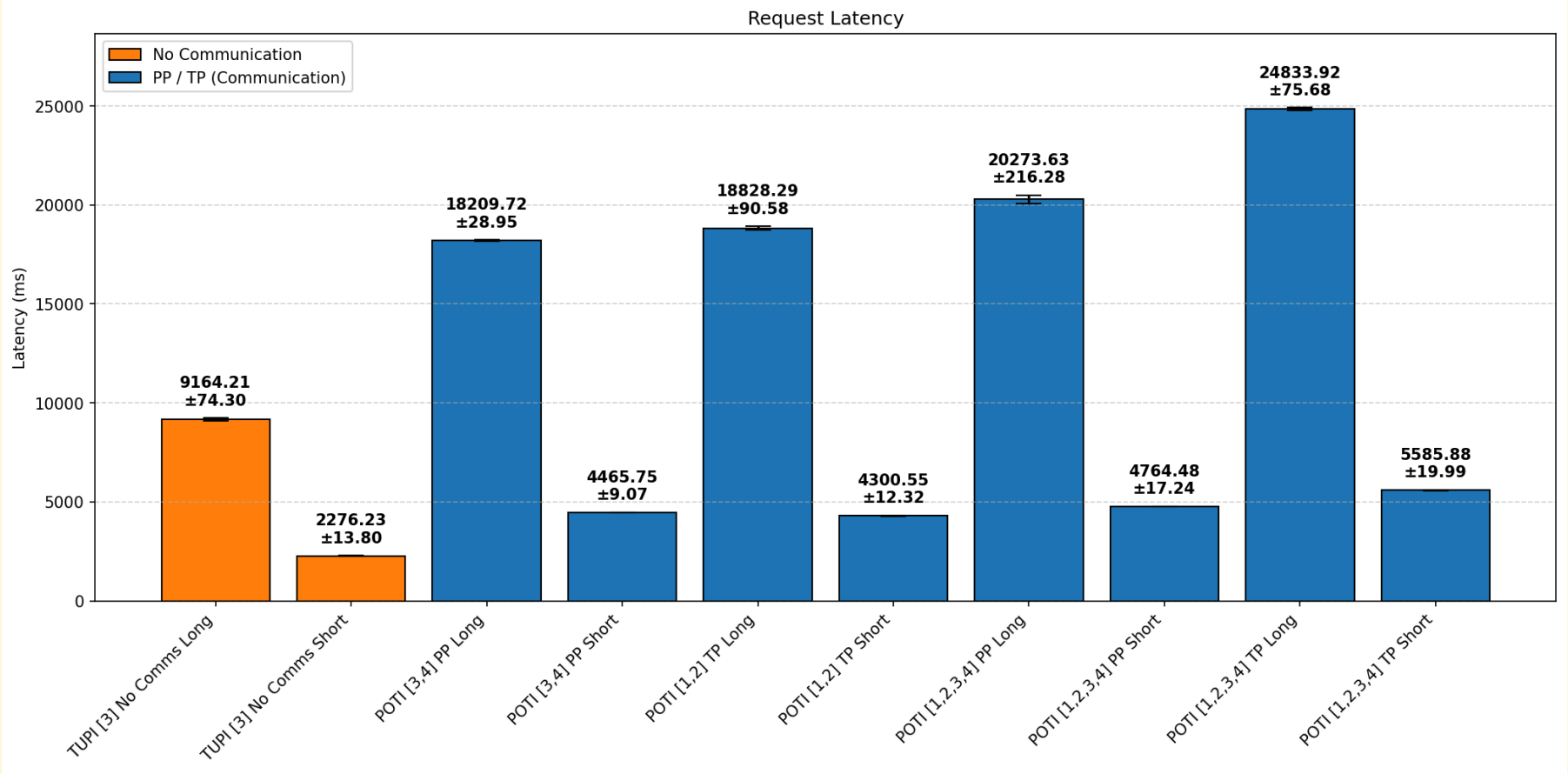
Principais métricas analisadas:

- **Request Latency**
- **Time to First Token (TTFT)**
- **Inter-Token Latency (ITL)**
- **Throughput (tokens/s)**

Observação geral:

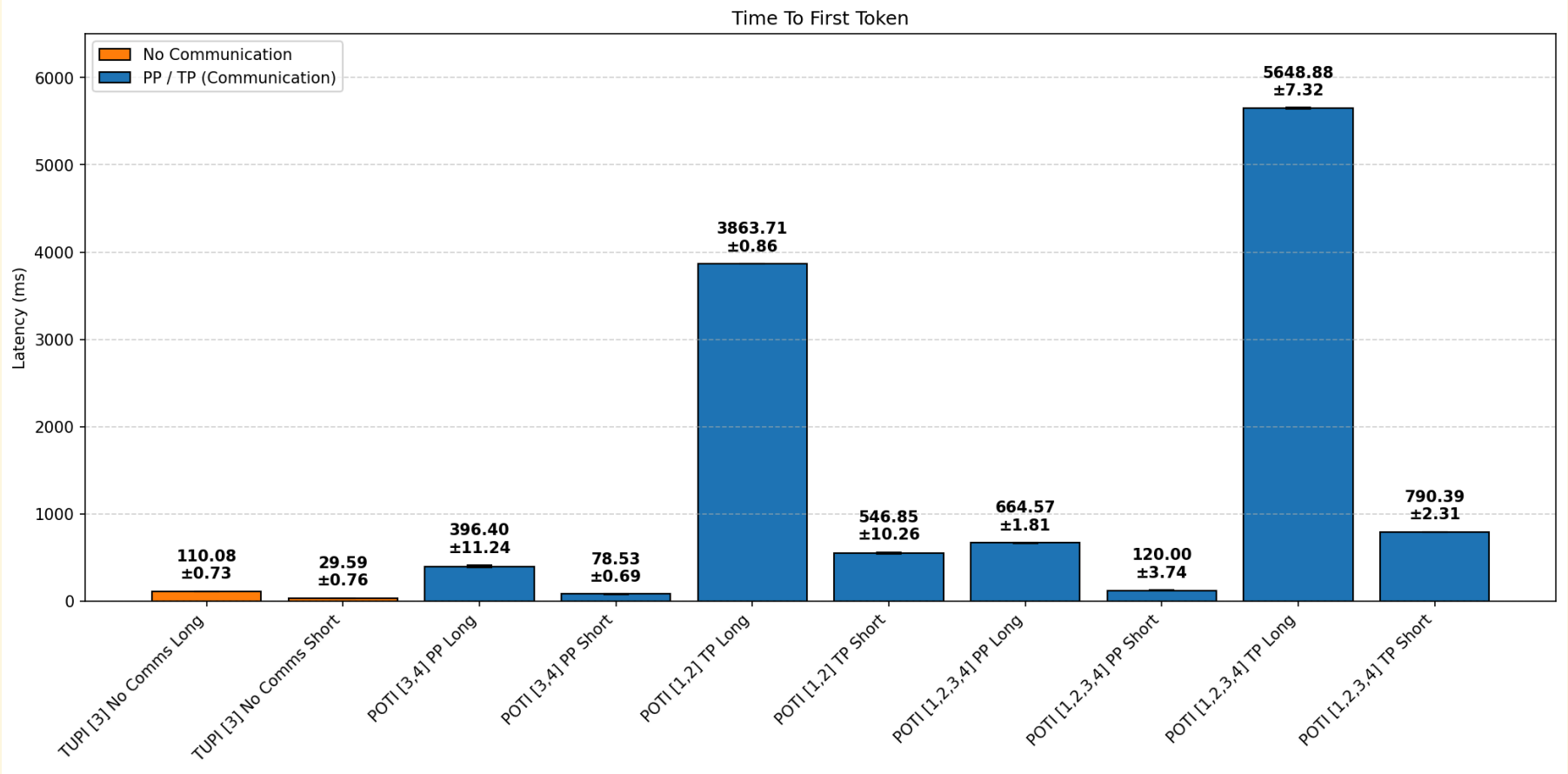
- Mais hardware ≠ melhor desempenho

Request Latency



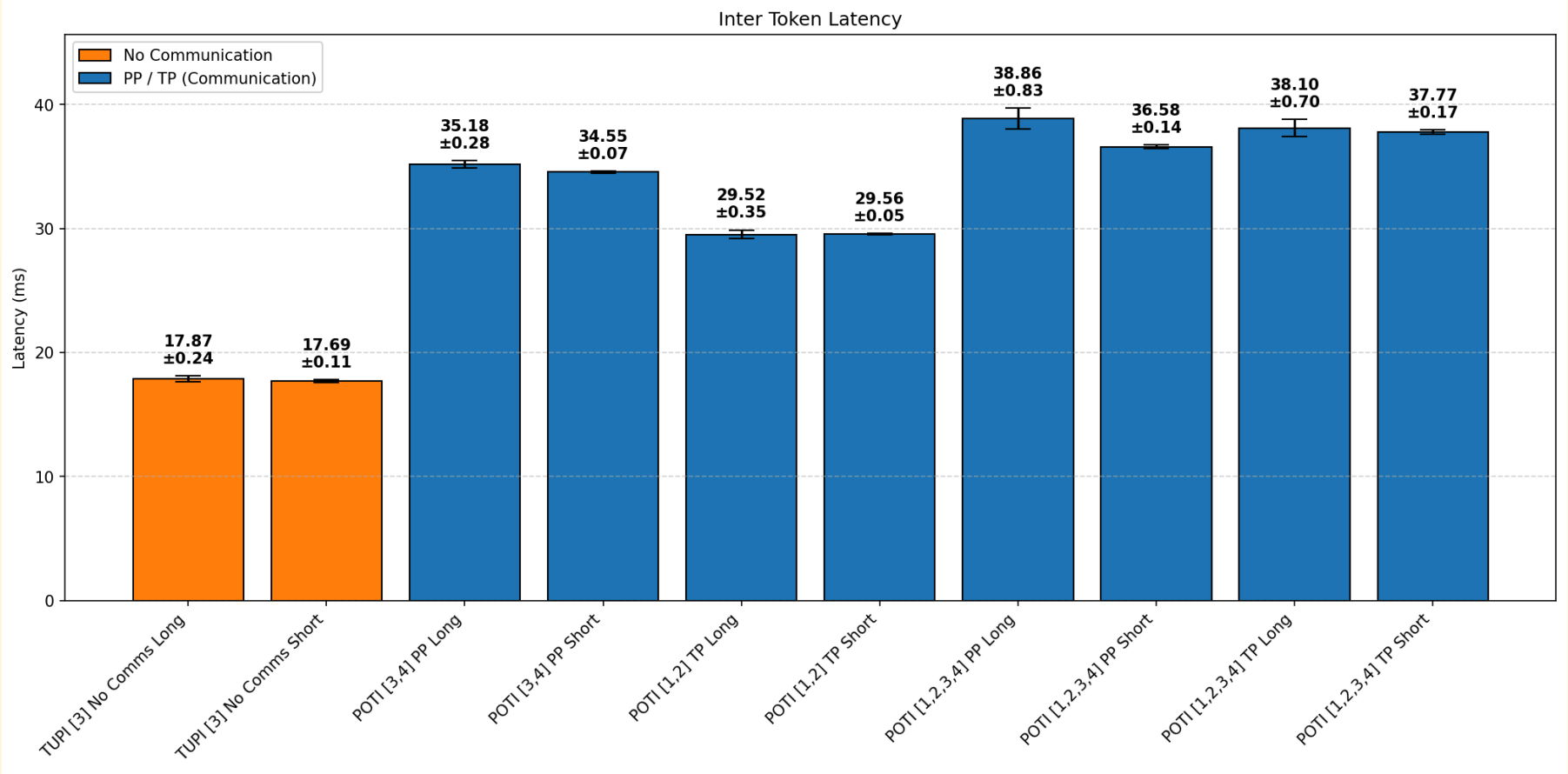
- TP no geral mais lento que o PP n otempo total
- Relação basicamente linear entre o tempo e tamanho do input

Time To First Token



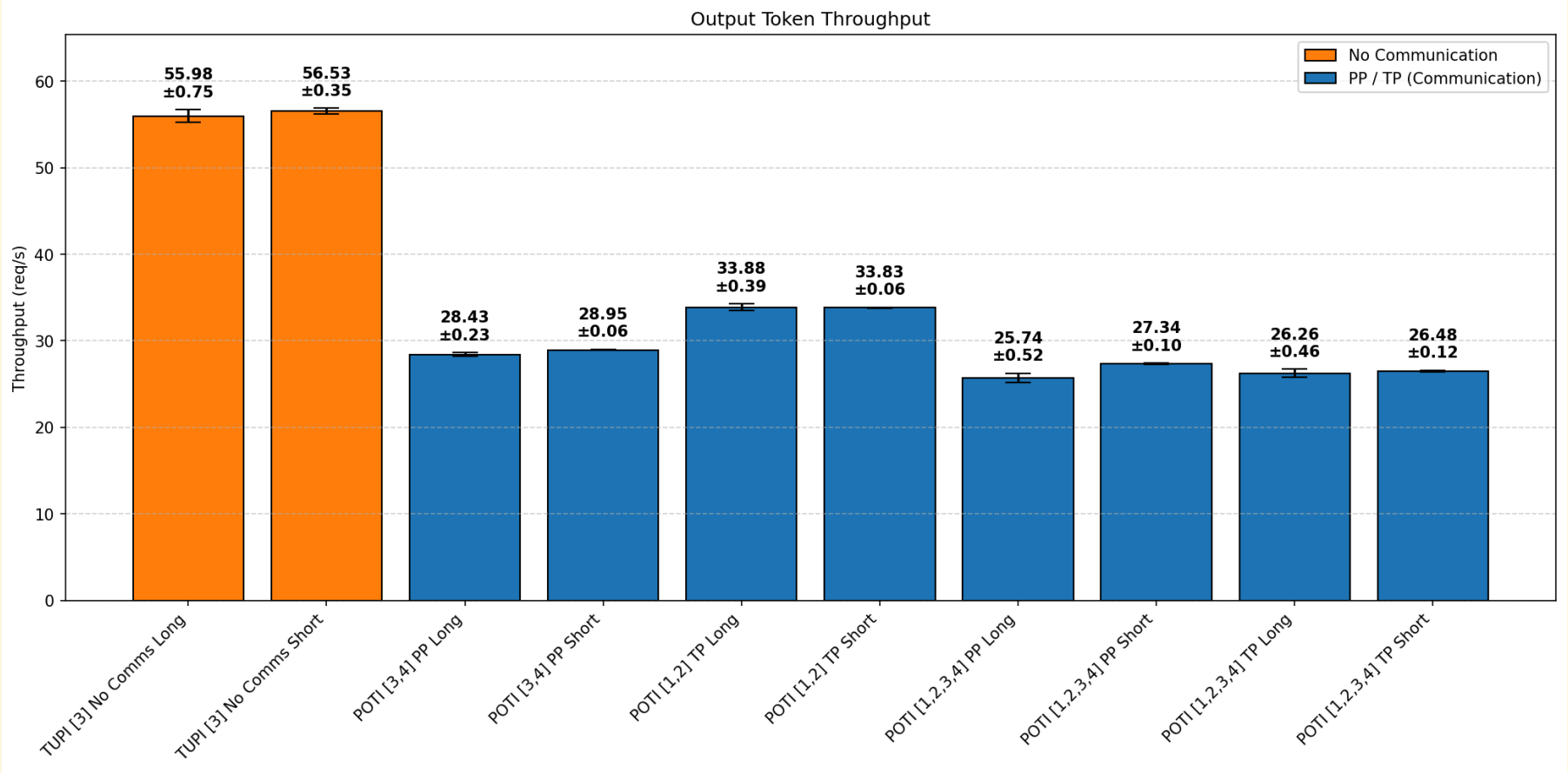
- TTFT do TP consideravelmente maior que os dos outros

Inter Token Latency



- TP com o menor tempo entre aqueles com comunicação

Output Token Throughput



- 1 GPU mais rápida

Análise — Inter-Token Latency

Resultados típicos:

- **Single GPU:** ~16 ms
- **TP:** intermediário
- **PP:** ~33 ms

Interpretação:

- ITL é altamente sensível à comunicação
- PP sofre com sincronização entre estágios

Análise — Time to First Token (TTFT)

Resultado contraintuitivo:

- **PP apresentou TTFT até 10x menor**

Explicação:

- Prefill pode ser parcialmente paralelizado
- Pipeline permite início antecipado do processamento

Conclusão:

- PP favorece início rápido, mas penaliza execução contínua

Trade-off Fundamental

Existe um conflito claro:

Métrica	Melhor abordagem
TTFT	Pipeline Parallelism
ITL	Single GPU
Latência total	Single GPU
Utilização GPU	Tensor Parallelism

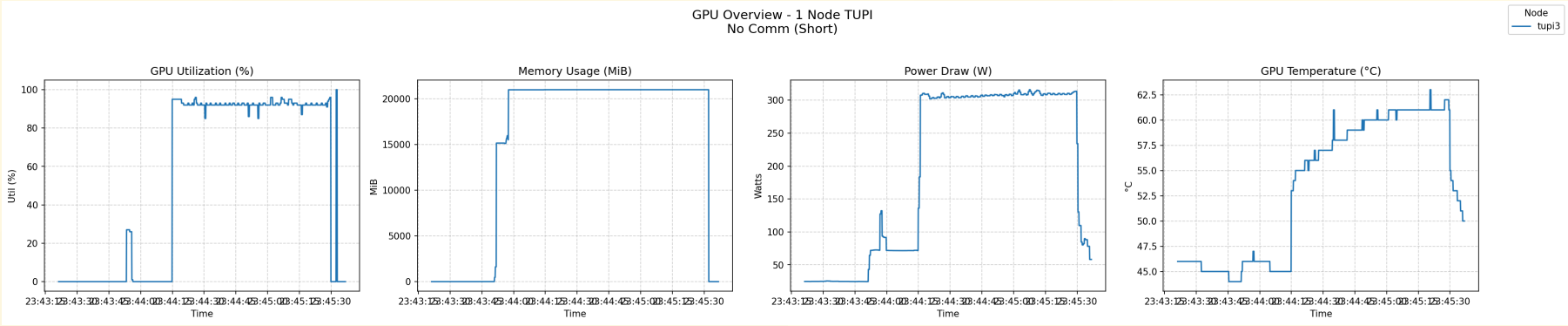
Comunicação como Gargalo

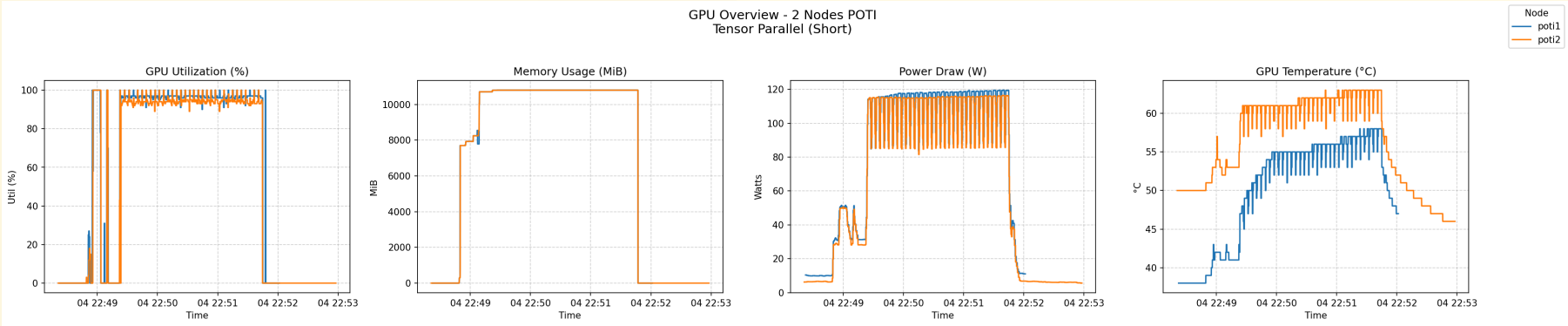
Diferença principal entre cenários:

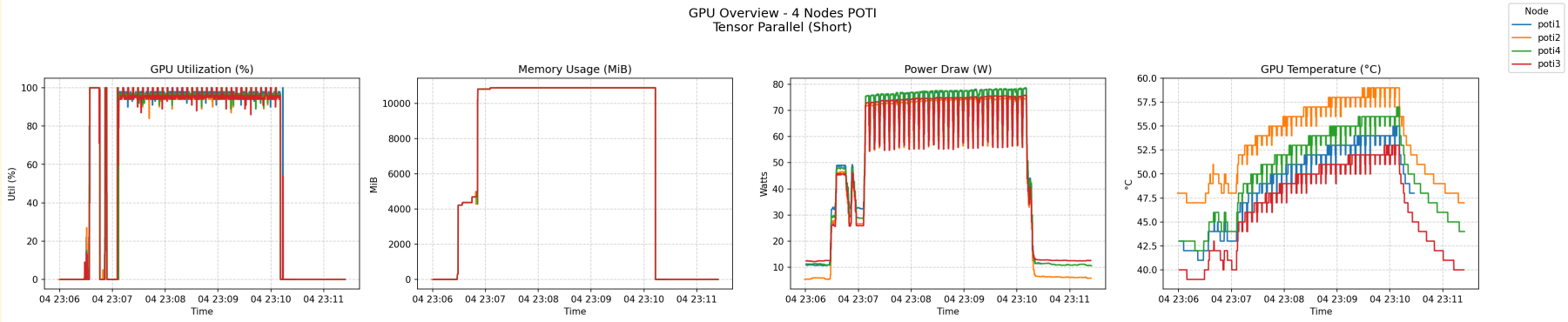
- **Sem comunicação:** execução local → mais eficiente
- **Com comunicação:**
  - sincronização
  - transferência de dados
  - latência de rede

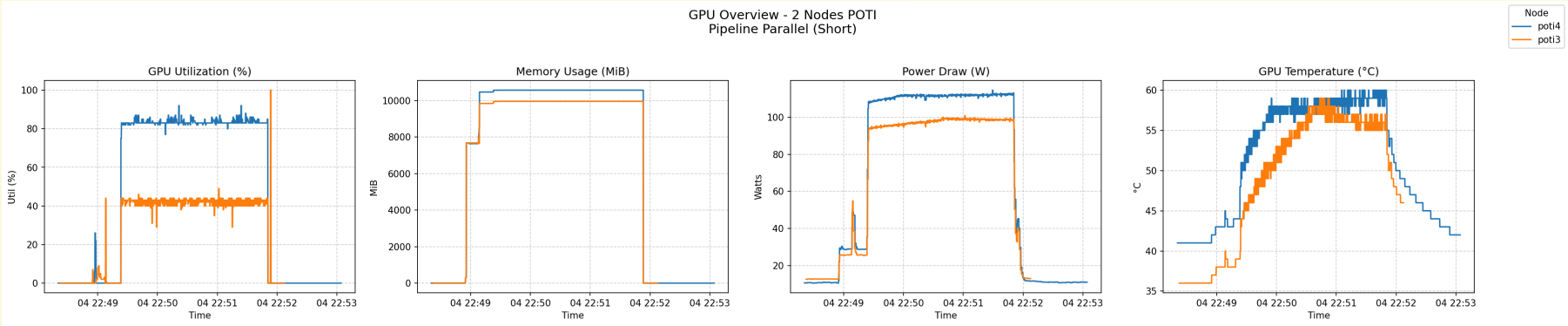
➡️
Comunicação domina o custo total

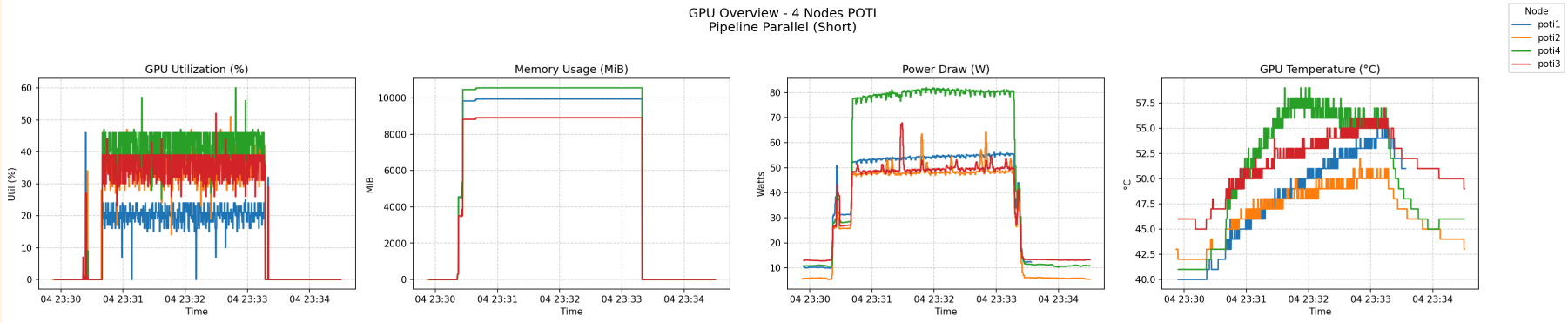
Métricas de GPU






GPU Overview - 4 Nodes POTI
Tensor Parallel (Short)
Node
poti1
poti2
poti4
poti3
GPU Utilization (%)
Util (%)
Memory Usage (MiB)
MiB
Power Draw (W)
Watts
GPU Temperature (°C)
°C
Time
04 23:06
04 23:07
04 23:08
04 23:09
04 23:10
04 23:11


GPU Overview - 2 Nodes POTI
Pipeline Parallel (Short)
Node
poti4
poti3
GPU Utilization (%)
Util (%)
Memory Usage (MiB)
MiB
Power Draw (W)
Watts
GPU Temperature (°C)
°C
Time
04 22:49
04 22:50
04 22:51
04 22:52
04 22:53


GPU Overview - 4 Nodes POTI
Pipeline Parallel (Short)
Node
poti1
poti2
poti4
poti3
GPU Utilization (%)
Util (%)
Memory Usage (MiB)
MiB
Power Draw (W)
Watts
GPU Temperature (°C)
°C
Time
04 23:30
04 23:31
04 23:32
04 23:33
04 23:34

Resultado Principal

Single GPU apresentou melhor desempenho geral

- Menor **latência total**
- Melhor **inter-token latency**
- Maior **throughput efetivo**

Interpretação:

- Ausência de comunicação elimina overhead

Tensor Parallelism (TP)

Características observadas:

- GPUs próximas de **100% de utilização**
- Boa distribuição de carga
- Overhead moderado de comunicação

Impacto:

- Latência maior que baseline
- Ainda eficiente em comparação com PP

Pipeline Parallelism (PP)

Comportamento identificado:

- Uma GPU ~**40–50%**
- Outra GPU ~**90–100%**

Problema:

- **Desbalanceamento de pipeline**
- Execução parcialmente sequencial

Consequência:

- Maior latência total
- Pior inter-token latency

Utilização das GPUs

Embora tenha sido mais eficiente no geral, a utilização de apenas 1 GPU pode representar um grande esforço concentrado em apenas uma única máquina, contribuindo para o seu desgaste mais rápido. A sua temperatura e uso de energia foram mais elevados que nos outros casos.

Dificuldades Encontradas

- Acesso limitado a algumas máquinas
- Inconsistência de recursos entre nós
- Presença de **tokens sensíveis em logs** (bloqueio de push)
- Ajuste manual de experimentos distribuídos

Soluções Aplicadas

- Troca de modelo (Qwen)
- Limpeza de logs e remoção de segredos
- Organização dos dados experimentais
- Padronização de métricas e scripts

Boas Práticas Adotadas

- Uso de **scripts reprodutíveis**
- Separação clara entre:
  - dados
  - código
  - resultados
- Controle de versões com Git
- Visualizações padronizadas

Ferramentas Utilizadas

- Python (pandas, matplotlib, seaborn)
- Jupyter Notebooks
- nvidia-smi (telemetria)
- Slurm (execução no cluster)
- Git + GitHub

Conclusão Parcial

- Comunicação é o principal fator limitante
- Mais GPUs nem sempre melhoram desempenho
- TP é mais eficiente que PP no cenário analisado
- Single GPU ainda é o melhor baseline quando possível

Próximos Passos

- Refinar medição de **tempo de comunicação**
- Separar claramente:
  - prefill vs decode
- Melhorar balanceamento no pipeline
- Avaliar escalabilidade com mais nós

Avaliação do Progresso

✓ Ambiente definido

✓ Métricas coletadas

✓ Resultados consistentes

✓ Gargalos identificados

Status: progresso consistente e metodologia validada